

บทที่ 4 นิพจน์และตัวดำเนินการต่าง ๆ (Expression & Operators)

4.1 นิพจน์ (Expression)

นิพจน์ (Expression) คือ ชุดของค่า (values) ตัวแปร (variables) ตัวดำเนินการ (operators) และ/หรือการเรียกใช้ฟังก์ชัน (function calls) ที่เมื่อประมวลผลแล้วจะให้ผลลัพธ์ออกมาเป็นค่าใด (Value) ค่าหนึ่งเสมอ

สรุบนิพจน์คือส่วนของโปรแกรมที่เมื่อคำนวณ หรือ ประมวลผลแล้วจะให้ผลลัพธ์ออกมาเป็นค่าได้

ตัวอย่างของนิพจน์ใน Python

```
a = 30 + 9
b = a + 5
print(b)
```

4.2 ตัวดำเนินการ (Operators)

ตัวดำเนินการ (Operators) คือ เครื่องหมายหรือสัญลักษณ์พิเศษต่าง ๆ ที่ใช้ประมวลผลข้อมูล เช่น

- + บวก (Addition)
- ลบ (Subtraction)
- * คูณ (Multiplication)
- / หาร (Division)

โดยค่าที่ถูกนำมาประมวลผลจะเรียกว่า ตัวถูกดำเนินการ (Operand)

4.3 ตัวถูกดำเนินการ (Operand)

ตัวถูกดำเนินการ (Operand) คือตัวถูกดำเนินการ หรือ ตัวถูกกระทำ อาจเป็นค่าคงที่ (Literals) ตัวแปร (Variables) นิพจน์ (Expressions) หรือ ฟังก์ชันที่ถูกเรียกใช้งาน (function call) ก็ได้

ตัวอย่าง

```
result = (5 * 2) + 7
```

- 5, 2, 7 เป็น ตัวถูกดำเนินการ (Operand)
- * (การคูณ) เป็น ตัวดำเนินการ (Operator)
- + (การบวก) เป็น ตัวดำเนินการ (Operator)
- ในบริบทที่กว้างขึ้น ($x * y$) ซึ่งเป็นนิพจน์ย่อย ก็สามารถทำหน้าที่เป็น ตัวถูกดำเนินการ สำหรับตัวดำเนินการ + ได้เช่นกัน

4.4 ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators)

ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators) คือ สัญลักษณ์ที่ใช้สำหรับดำเนินการคำนวณทางคณิตศาสตร์กับข้อมูลประเภทตัวเลข เช่น การบวก ลบ คูณ หาร ยกกำลัง

ตัวดำเนินการเหล่านี้เป็นส่วนสำคัญของการสร้างนิพจน์ (expression) เพื่อให้ได้ผลลัพธ์เชิงตัวเลขในโปรแกรม

ตาราง 4 ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators)

ตัวดำเนินการ	ชื่อเรียก	ความหมาย	ตัวอย่าง	ผลลัพธ์
+	บวก (Addition)	การบวก	$5 + 3$	8
-	ลบ (Subtraction)	การลบ	$10 - 4$	6
*	คูณ (Multiplication)	การคูณ	$6 * 7$	42
/	หาร (Division)	การหาร (ได้ผลลัพธ์เป็นทศนิยมเสมอ)	$10 / 3$	3.3333333 33333335
//	หารปัดเศษ (Floor Division)	การหารที่ได้ผลลัพธ์เป็นจำนวนเต็ม (ปัดเศษทิ้งลง)	$10 // 3$	3
%	หารเอาเศษ (Modulo)	การหารที่ได้ผลลัพธ์เป็นเศษ	$10 \% 3$	1
**	ยกกำลัง (Exponentiation)	การยกกำลัง	$2 ** 3$	8

4.5 ตัวดำเนินการเปรียบเทียบ (Comparison Operators)

ตัวดำเนินการเปรียบเทียบ (Comparison Operators) หรือบางครั้งเรียกว่า ตัวดำเนินการเชิงสัมพันธ์ (Relational Operators) คือสัญลักษณ์ที่ใช้ในการประเมินความสัมพันธ์ระหว่างค่าข้อมูลสองค่า (operands) โดยจะให้ผลลัพธ์เป็น ค่าตรรกะ (Boolean Value) เสมอ ซึ่งได้แก่ True หรือ False

ตัวดำเนินการเหล่านี้มีความสำคัญอย่างยิ่งในการสร้างเงื่อนไข (Conditions) และการควบคุมการไหลของโปรแกรม (Control Flow) อาทิ ในคำสั่งเลือกทำ (if, elif, else) หรือลูป (while) เพื่อใช้ในการตัดสินใจว่าโปรแกรมควรดำเนินการตามชุดคำสั่งใด

ตาราง 5 ตัวดำเนินการเปรียบเทียบ (Comparison Operators)

ตัวดำเนินการ	ชื่อเรียก	ความหมาย	ตัวอย่าง	ผลลัพธ์
==	เท่ากับ (Equal to)	ตรวจสอบว่าค่าทั้งสองฝั่ง เท่ากันหรือไม่	5 == 5	True
			5 == 3	False
			"apple" == "apple"	True
!=	ไม่เท่ากับ (Not equal to)	ตรวจสอบว่าค่าทั้งสองฝั่ง ไม่เท่ากันหรือไม่	5 != 3	True
			5 != 5	False
			"apple" != "orange"	True
>	มากกว่า (Greater than)	ตรวจสอบว่าค่าฝั่งซ้ายมากกว่า ค่าฝั่งขวาหรือไม่	10 > 5	True
			5 > 10	False
<	น้อยกว่า (Less than)	ตรวจสอบว่าค่าฝั่งซ้าย น้อยกว่าค่าฝั่งขวาหรือไม่	5 < 10	True
			10 < 5	False
>=	มากกว่าหรือเท่ากับ (Greater than or equal to)	ตรวจสอบว่าค่าฝั่งซ้ายมากกว่า หรือเท่ากับค่าฝั่งขวาหรือไม่	10 >= 10	True
			10 >= 5	True
			5 >= 10	False
<=	น้อยกว่าหรือเท่ากับ (Less than or equal to)	ตรวจสอบว่าค่าฝั่งซ้ายน้อยกว่า หรือเท่ากับค่าฝั่งขวาหรือไม่	5 <= 5	True
			5 <= 10	True
			10 <= 5	False

4.6 ตัวดำเนินการทางตรรกศาสตร์ (Logical Operators)

ตัวดำเนินการทางตรรกศาสตร์ (Logical Operators) คือสัญลักษณ์ที่ใช้ในการเชื่อมโยงหรือประมวลผลนิพจน์เชิงตรรกะ (Boolean Expressions) หรือค่าตรรกะ (Boolean Values True และ False) เพื่อให้ได้ผลลัพธ์สุดท้ายออกมาเป็นค่าตรรกะเพียงค่าเดียว (True หรือ False)

ตัวดำเนินการเหล่านี้เป็นส่วนสำคัญในการสร้างเงื่อนไขที่ซับซ้อนและควบคุมการไหลของโปรแกรม (Control Flow) ในภาษาโปรแกรม

ตัวดำเนินการทางตรรกศาสตร์แบ่งเป็น 3 ประเภท ดังนี้

1. ตัวดำเนินการ and (และ) จะให้ผลลัพธ์เป็น True ก็ต่อเมื่อนิพจน์ที่เชื่อมโยงกันเป็น True ทั้งหมด หรือ ตัวถูกดำเนินการ A และ B เป็น True ทั้งคู่เท่านั้น

ตาราง 6 ตัวดำเนินการ and (และ)

A	B	A and B
True	True	True
True	False	False
False	TRUE	False
False	False	False

2. ตัวดำเนินการ or (หรือ) จะให้ผลลัพธ์เป็น True เมื่อมีนิพจน์ใดนิพจน์หนึ่งเป็น True หรือ ตัวถูกดำเนินการ A และ B เป็น False ทั้งคู่เท่านั้น ถึงจะให้ผลลัพธ์เป็น False

ตาราง 7 ตัวดำเนินการ or (หรือ)

A	B	A and B
True	True	True
True	False	True
False	TRUE	True
False	False	False

3. ตัวดำเนินการ not (นิเสธ) เป็นตัวดำเนินการที่จะเปลี่ยนค่า True เป็น False และ False เป็น True

ตาราง 8 ตัวดำเนินการ not (นิเสธ)

A	not A
True	False
False	True

4.7 ตัวดำเนินการระดับบิต (Bitwise Operators)

ตัวดำเนินการระดับบิต (Bitwise Operators) คือตัวดำเนินการที่ใช้ในการประมวลผลหรือเปรียบเทียบค่าข้อมูลในระดับ บิต (binary digit) ซึ่งเป็นการทำงานโดยตรงกับการแทนค่าฐานสอง (binary representation) ของตัวเลขจำนวนเต็ม

ตาราง 9 ตัวดำเนินการระดับบิต (Bitwise Operators)

ตัวดำเนินการ (Operator)	ชื่อ	คำอธิบาย
&	AND	กำหนดแต่ละบิตเป็น 1 หากบิตทั้งสอง (ของตัวถูกดำเนินการ) เป็น 1
	OR	กำหนดแต่ละบิตเป็น 1 หากบิตใดบิตหนึ่งหรือทั้งสองบิตเป็น 1
^	XOR	กำหนดแต่ละบิตเป็น 1 หากบิตเพียงบิตเดียวเท่านั้นที่เป็น 1
~	NOT	กลับบิตทั้งหมด (เปลี่ยน 1 เป็น 0 และ 0 เป็น 1)
<<	Zero-fill Left Shift (การเลื่อนบิตซ้ายแบบเติมศูนย์)	เลื่อนบิตไปทางซ้าย โดยเติมศูนย์จากทางขวา และปล่อยให้บิตซ้ายสุดหลุดออก (ถูกละทิ้ง)
>>	Signed Right Shift (การเลื่อนบิตขวาแบบคงเครื่องหมาย)	เลื่อนบิตไปทางขวา โดยเติมสำเนาของบิตซ้ายสุด (Sign Bit) จากทางซ้าย และปล่อยให้บิตขวาสุดหลุดออก (ถูกละทิ้ง)

4.8 ตัวดำเนินการกำหนดค่า (Assignment Operators)

ตัวดำเนินการกำหนดค่า (Assignment Operator) คือ สัญลักษณ์ที่ใช้สำหรับกำหนดค่าให้กับตัวแปรในโปรแกรม โดยทั่วไปใช้เครื่องหมายเท่ากับ (=) การทำงานของตัวดำเนินการกำหนดค่า คือ นำค่าหรือผลลัพธ์ที่อยู่ทางด้านขวาของเครื่องหมายเท่ากับ ไปกำหนดให้กับตัวแปรที่อยู่ทางด้านซ้าย ซึ่งค่าทางด้านขวาสามารถเป็นค่าคงที่ ตัวแปร หรือ นิพจน์ (expression) ก็ได้

นอกจากนี้ ยังมีตัวดำเนินการกำหนดค่าแบบย่อ (Compound Assignment Operators) เป็นการรวมการดำเนินการ (เช่น บวก, ลบ, คูณ) เข้ากับการกำหนดค่า (=) เพื่อเขียนโค้ดให้สั้นลงและอ่านง่ายขึ้น

ตาราง 10 ตัวดำเนินการกำหนดค่า (Assignment Operators)

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	เทียบเท่ากับ
=	กำหนดค่าให้กับตัวแปร	$x = 5$	$x = 5$
+=	บวกค่าก่อนแล้วกำหนดค่าให้กับตัวแปร	$x += 5$	$x = x + 5$
-=	ลบค่าก่อนแล้วกำหนดค่าให้กับตัวแปร	$x -= 3$	$x = x - 3$
*=	คูณค่าก่อนแล้วกำหนดค่าให้กับตัวแปร	$x *= 2$	$x = x * 2$
/=	หารค่าก่อนแล้วกำหนดค่าให้กับตัวแปร	$x /= 4$	$x = x / 4$
%=	หารเอาเศษก่อนแล้วกำหนดค่าให้กับตัวแปร	$x \% = 3$	$x = x \% 3$
**=	ยกกำลังก่อนแล้วกำหนดค่าให้กับตัวแปร	$x ** = 2$	$x = x ** 2$
//=	หารปัดเศษก่อนแล้วกำหนดค่าให้กับตัวแปร	$x //= 2$	$x = x // 2$

4.9 ตัวดำเนินการเอกลักษณ์ (Identity Operators)

ตัวดำเนินการเอกลักษณ์ (Identity Operators) ในภาษาไพธอน คือ ตัวดำเนินการที่ใช้สำหรับตรวจสอบว่า ตัวแปรสองตัวชี้ไปยังวัตถุเดียวกันในหน่วยความจำหรือไม่ กล่าวคือ เปรียบเทียบอัตลักษณ์ของวัตถุ (object identity) ไม่ใช่แค่เปรียบเทียบค่าของวัตถุเหมือนกันหรือไม่ ตัวดำเนินการเอกลักษณ์นี้จะคืนค่าเป็นบูลีน คือ True หากทั้งสองตัวแปรชี้ไปยังวัตถุเดียวกัน และ False หากไม่ใช่

ตัวดำเนินการเอกลักษณ์ในภาษาไพธอนมี 2 ตัว ได้แก่

1. `is` ใช้ตรวจสอบว่า ตัวแปรซ้ายและขวาชี้ไปยังวัตถุเดียวกันหรือไม่
2. `is not` ใช้ตรวจสอบว่า ตัวแปรซ้ายและขวาไม่ชี้ไปยังวัตถุเดียวกันหรือไม่

ตัวอย่าง

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a is b)    # ผลลัพธ์: True เพราะ b ชี้ไปยังวัตถุเดียวกับ a
print(a is c)    # ผลลัพธ์: False เพราะ c เป็นวัตถุคนละตัวกับ a ถึงค่าจะเหมือนกัน
print(a == c)    # ผลลัพธ์: True เพราะค่าของ a และ c เท่ากัน
print(a is not c) # ผลลัพธ์: True เพราะ a และ c ไม่ใช่วัตถุเดียวกัน
```

4.10 ตัวดำเนินการสมาชิก (Membership Operators)

ตัวดำเนินการสมาชิก (Membership Operators) คือ ตัวดำเนินการที่ใช้สำหรับตรวจสอบว่าค่าหรือวัตถุหนึ่ง ๆ เป็นสมาชิก (member) ของลำดับข้อมูลประเภทต่าง ๆ หรือไม่ เช่น string, list, tuple หรือ dictionary หรือไม่ โดยผลลัพธ์ของตัวดำเนินการนี้จะเป็นค่าบูลีน คือ True หากเป็นสมาชิก และ False หากไม่เป็นสมาชิก

ตัวดำเนินการสมาชิกหลักในภาษาไพธอนมี 2 ตัว ได้แก่

1. `in` (อยู่ใน / เป็นสมาชิกของ) จะตรวจสอบว่าค่าที่อยู่ทางซ้ายมือ มีอยู่ในลำดับข้อมูลที่อยู่ทางขวามือหรือไม่ โดยจะคืนค่า True หากค่าที่ตรวจสอบเป็นสมาชิกของลำดับข้อมูลนั้น
2. `not in` (ไม่อยู่ใน / ไม่ใช่สมาชิกของ) จะตรวจสอบว่าค่าที่อยู่ทางซ้ายมือ ไม่อยู่ในลำดับข้อมูลที่อยู่ทางขวามือหรือไม่ โดยจะคืนค่า True หากค่าที่ตรวจสอบไม่เป็นสมาชิกของลำดับข้อมูลนั้น

ตัวดำเนินการสมาชิกนี้มีประโยชน์อย่างมากในการตรวจสอบการมีอยู่ของข้อมูลในโครงสร้างข้อมูลต่าง ๆ เพื่อใช้ควบคุมการทำงานของโปรแกรม เช่น การตัดสินใจเลือกทำงานตามเงื่อนไข

ตัวอย่างการใช้ตัวดำเนินการสมาชิก (Membership Operators)

```
# ในสายอักขระ (String)
text = "Hello Python"
print("Python" in text)  # ผลลัพธ์: True (เพราะ "Python" อยู่ใน "Hello Python")
print("Java" in text)    # ผลลัพธ์: False (เพราะ "Java" ไม่อยู่ใน "Hello Python")
print("o" in text)       # ผลลัพธ์: True
print("Java" not in text) # ผลลัพธ์: True
```

4.11 ลำดับความสำคัญของตัวดำเนินการ (Operator Precedence)

ลำดับความสำคัญของตัวดำเนินการ (Operator Precedence) คือ กฎเกณฑ์ที่กำหนดว่าตัวดำเนินการใดในนิพจน์ (expression) จะถูกประมวลผลก่อน-หลัง หากมีตัวดำเนินการในนิพจน์ที่ประกอบด้วยตัวดำเนินการหลายประเภทหรือตัวดำเนินการประเภทเดียวกันรวมอยู่ในนิพจน์เดียวกัน การทำความเข้าใจลำดับความสำคัญนี้มีความสำคัญอย่างยิ่งในการเขียนโปรแกรมที่ถูกต้อง แม่นยำ และคาดเดาผลลัพธ์ได้

พิจารณานิพจน์ $5 + 2 * 3$

- ถ้าคำนวณ + ก่อน จะได้ $(5 + 2) * 3 = 7 * 3 = 21$
- ถ้าคำนวณ * ก่อน จะได้ $5 + (2 * 3) = 5 + 6 = 11$

ผลลัพธ์ที่ได้แตกต่างกันอย่างชัดเจน แสดงให้เห็นว่าลำดับการคำนวณมีความสำคัญ

ตาราง 11 ลำดับความสำคัญของตัวดำเนินการในภาษาไพธอนโดยเรียงลำดับจากสูงไปต่ำ

ลำดับความสำคัญ	ตัวดำเนินการ	ความหมาย
1	()	วงเล็บ ใช้กำหนดลำดับการประมวลผลก่อนเสมอ
2	**	ตัวดำเนินการทางคณิตศาสตร์ ยกกำลัง
3	~,+,-	ตัวดำเนินการระดับบิต Complement, unary plus and minus
4	*, /, %, //	ตัวดำเนินการทางคณิตศาสตร์ คูณหารหารเอาเศษ (modulo) หารปัดเศษ (floor division)

ลำดับความสำคัญ	ตัวดำเนินการ	ความหมาย
5	+, -	ตัวดำเนินการทางคณิตศาสตร์ บวก ลบ
6	>>, <<	ตัวดำเนินการระดับบิต เลื่อนบิตขวา ซ้าย (Bitwise shift)
7	&	ตัวดำเนินการระดับบิต and
8	^,	ตัวดำเนินการระดับบิต xor, or
9	<=, <, >, >=	ตัวดำเนินการเปรียบเทียบ (น้อยกว่าหรือเท่ากับ น้อยกว่า มากกว่า มากกว่าหรือเท่ากับ)
10	=, !=	ตัวดำเนินการเปรียบเทียบ เท่ากับ ไม่เท่ากับ
11	=, %=, /=, //, -=, +=, *=, **=	ตัวดำเนินการกำหนดค่าและกำหนดค่าแบบร่วม
12	is, is not	ตัวดำเนินการตรวจสอบอัตลักษณ์ (identity)
13	in, not in	ตัวดำเนินการตรวจสอบสมาชิก (membership)
14	not, or, and	ตัวดำเนินการตรรกะ (logical operators)

หมายเหตุ

- วงเล็บ () มีลำดับความสำคัญสูงสุด ใช้บังคับให้ประมวลผลภายในวงเล็บก่อนเสมอ
- ตัวดำเนินการที่มีลำดับความสำคัญเท่ากันจะประมวลผลจากซ้ายไปขวา (ยกเว้น ** ที่ประมวลผลจากขวาไปซ้าย)